

BRANCHBENCH: An Extensible Benchmark for Agentic Database Branching

Elaine Ang
Columbia University
ra3448@columbia.edu

Kevin Durand
Columbia University
kpd2136@columbia.edu

Sam Weldon
Columbia University
sw3927@columbia.edu

Kostis Kaffes
Columbia University
kkaffes@cs.columbia.edu

In Keun Kim
Columbia University
ik2619@columbia.edu

Eugene Wu
Columbia University
ewu@cs.columbia.edu

ABSTRACT

Agentic systems increasingly use databases as persistent working memory for speculative action: they fork state, mutate it, evaluate outcomes, and discard or retain candidate trajectories. Existing relational benchmarks assume a single shared state and therefore do not measure the branch lifecycle, branch-tree scalability, cross-branch comparison, or resource reclamation costs induced by these workloads. We present BRANCHBENCH, an extensible benchmark framework that instantiates branch-mutate-evaluate workloads from explicit topology, mutation, query, and pruning parameters. BRANCHBENCH enables users to synthesize benchmark suites with workload-specific branching characteristics: macrobenchmarks instantiate parameterized workflows, while microbenchmarks isolate individual operations to explain their performance characteristics. We demonstrate BRANCHBENCH’s usability on Neon and Dolt, two state-of-the-art branchable DBMSes with contrasting architectures, and show how the benchmark exposes the tension between branch agility and branch-local query performance.

1 INTRODUCTION

Large language model agents are changing database workloads from linear transaction streams into *agentic speculation*: repeated construction and evaluation of hypothetical states. These workloads interleave frequent branch creation, isolated schema and data mutations, branch-local read-heavy evaluation, cross-branch comparison, and aggressive pruning over wide or deep exploration trees. As an example, Monte Carlo Tree Search (MCTS) instantiates this access pattern by selecting, expanding, and scoring branches over database-backed state. MCTS has already improved Terminal-Bench success rates from 3.4% to 30.6% by efficiently exploring alternative action trajectories [5, 10]. These gains require the data system to manage many isolated speculative states, not merely to execute queries efficiently within one state.

Traditional DBMS mechanisms are poorly matched to this access pattern. Transactions and savepoints provide rollback within a linear execution context, but they do not provide persistent, named, independently mutable states that can be revisited, compared, and pruned across long-running exploration. Physical clones provide isolation but copy or reserve resources at a granularity too coarse for high-frequency exploration.

Recent branchable databases expose Git-like operations over relational state. Neon branches by sharing WAL-derived page histories [8]; TigerData uses distributed block storage to create copy-on-write branches [4]; Xata uses block-level copy-on-write below

PostgreSQL [9]; and Dolt represents database state with a content-addressed Merkle/prolly-tree storage engine [7]. These mechanisms differ in copy-on-write granularity, metadata organization, compute allocation, garbage collection, and support for diff and merge.

The relevant question is therefore not whether a DBMS exposes a branch command, but whether its branching design matches an agentic workload. Concrete agents may induce very different access patterns: one agent may explore schema-changing branches with backfills, another may replay alternative transaction prefixes, and another may build a deep speculative tree with hundreds or thousands of states. These examples are illustrative rather than exhaustive; even agents in the same application domain can differ substantially in fanout, depth, mutation mix, read intensity, and pruning behavior. A useful benchmark must therefore parameterize the shape and lifecycle of speculative state, rather than hard-code a small set of agent categories.

Existing database benchmarks parameterize operations over one logical database state, not over a branching state space. TPC-C and TPC-H define standard transactional and analytical workloads, respectively; CH-benCHmark combines them to evaluate hybrid OLTP/OLAP execution over one logical state [1]. YCSB parameterizes request mixes for cloud serving systems, and OLTP-Bench provides configurable OLTP workloads, but both execute against a single evolving database instance rather than a tree-structured set of branch states [2, 3]. Similar to SEQUOIA 2000, which exposed how scientific workloads exceeded the assumptions of prior benchmarks [6], agentic speculation requires a benchmark that makes branching structure a first-class workload parameter.

We present BRANCHBENCH¹, an extensible benchmark framework for evaluating branchable relational DBMSes under agentic workloads. BRANCHBENCH synthesizes workflows by instantiating a common branch-mutate-evaluate-prune loop with explicit parameters for worker count, branch fanout, branch depth, schema mutations, data queries, cross-branch comparisons, and pruning. Users can add new backends behind a small branch API and add new workloads by supplying parameter configurations, schema files, and SQL operation templates.

This short paper makes three contributions:

- We define agentic database branching as a benchmark target and identify workload dimensions that are absent from conventional relational benchmarks.

¹Benchmark code: <https://github.com/ElaineAng/db-fork>.

Group	Parameter	Software Dev	Failure Repro	Data Cleaning	MCTS	MC Simulation
Setup	Workers T	5	1	10	10	1000
	Steps/worker S	20	10	20	100	1
	Cross-branch C	1	—	2	—	1
	Root fanout F_r	5	10	10	10	1000
	Inner fanout F_i	3	—	3	10	—
	Depth D	3	1	3	25	1
Per-step	Schema changes M_s	1	5	1	—	—
	Data mutations M_d	1	45	1	1	50
	Read queries Q_v	2	1	1	1	1
	Prune prob. γ	0.1	1	—	0.1	1

Table 1: Representative BRANCHBENCH macrobenchmark instantiations. Setup parameters define the run shape and scale; per-step parameters define the work performed at each step; — means the operation is absent.

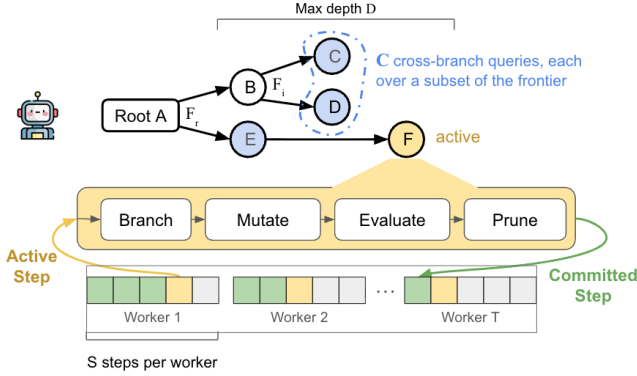


Figure 1: The BRANCHBENCH macrobenchmark executes a parameterized branch-mutate-evaluate-prune loop over a shared branch tree.

- We design BRANCHBENCH, an extensible macrobenchmark and microbenchmark framework for instantiating benchmark suites with workload-specific branch topology, mutation intensity, evaluation, and pruning characteristics.
- We demonstrate BRANCHBENCH by evaluating Neon and Dolt, two state-of-the-art branchable DBMSes with contrasting architectures. We show that Neon is up to 25× slower to create branches and 1500× slower to switch branches than Dolt, while Dolt’s range reads and joins become up to 4000× slower than Neon’s on read-heavy operations that scan or join larger data ranges.

2 BENCHMARK FRAMEWORK DESIGN

BRANCHBENCH separates the benchmark into a reusable execution engine, workload instantiations, and backend adapters. This structure is intended to make the benchmark extensible along two axes: users can add new agentic workloads by changing workload parameters, schema files, and SQL templates, and can add new systems by implementing the backend branch API; thus, the benchmark does not prescribe a fixed database schema; the schema is part of the workload definition.

Execution model. Figure 1 shows the macrobenchmark driver. The benchmark starts from a root branch and launches T workers,

each executing S steps over a shared branch tree. Tree shape is controlled by maximum root fanout F_r , maximum inner fanout F_i , and maximum depth D . These parameters cover flat stars, wide shallow trees, bushy trees, and deep search trajectories.

Each step has four phases. In the *branch* phase, the worker selects an eligible parent, creates a child branch, and connects to it. In the *mutate* phase, it applies M_s schema mutations and M_d data mutations. In the *evaluate* phase, it executes Q_v read queries that validate or score the branch state. In the *prune* phase, it deletes the branch with probability γ . Workloads may also issue C cross-branch queries over committed frontier leaves, capturing comparisons across candidate branch states or aggregate outcomes across simulations.

Table 1 summarizes the workload instantiations used in our evaluation. The goal is not to restrict agentic workloads to this specific set of parameters; rather, these configurations demonstrate how a user can translate a plausible workload hypothesis into a concrete BRANCHBENCH benchmark instance. For example, the table includes moderate branching with schema evolution (Software Dev), flat replay branches with heavy data operations (Failure Repro), alternative data-cleaning branches (Data Cleaning) compared by validation queries, deep search trees (MCTS), and wide short-lived fanout patterns (MC Simulation).

Backend interface and microbenchmarks. BRANCHBENCH abstracts each backend behind timed operations for **branch creation**, **branch connection**, **branch deletion**, and **SQL execution**. Both macrobenchmarks and microbenchmarks use this interface, so measurements are comparable across end-to-end workflows and isolated operation studies. Macrobenchmarks compose these operations into full branch-mutate-evaluate-prune executions and report completion time, branch-management fraction, and storage overhead. Microbenchmarks reuse the same operations in scripted setup and timed phases to isolate branch lifecycle costs, data and schema operations, and single- or multi-threaded execution effects. This shared instrumentation lets BRANCHBENCH attribute workflow-level slowdowns to specific measured components, such as branch lifecycle latency, schema mutation cost, read-query latency, or contention under concurrent execution.

3 EVALUATION

We use BRANCHBENCH to evaluate Neon and Dolt, two state-of-the-art branchable DBMSes that are sufficiently complete to run

System	Software Dev	Failure Repro	Data Cleaning	MCTS	MC Simulation
Neon	~	✓	~	~	✗
Dolt	✓	✓	✓	~	✗

Table 2: Full-workflow capability for the two demonstration systems within a two-hour timeout. ✓ indicates completion, ✗ indicates inability to execute the workload, and ~ indicates partial execution or timeout.

System	Software Dev	Failure Repro	Data Cleaning	MCTS	MC Simulation
Dolt	93.1 MB	63.6 MB	94.1 MB	3.8 MB	—
Neon	4.0 GB	0 B	4.7 MB	4.1 MB	—

Table 3: Storage overhead per workflow for Dolt and Neon.

representative branch–mutate–evaluate workflows while embodying different architectural choices. Neon is a hosted PostgreSQL-compatible system that allocates dedicated compute to branches; Dolt is a self-hosted relational system that stores versions in a content-addressed Merkle/prolly-tree representation. We evaluate the systems as available: Dolt was built from source and ran on a c8i.4xlarge EC2 instance in us-east-1, while Neon was evaluated through its hosted service in the same region.

This setup does not place the systems in identical environments: Neon includes network and hosted-service effects, while Dolt runs on a controlled EC2 instance. We could not obtain a stable source-built Neon deployment, and adding artificial delay to Dolt would address only one artifact. This limitation affects absolute latencies, but not the dominant effects below: branch lifecycle overheads, range-query degradation, and workflow completion failures are much larger than point-read network effects. A system owner could rerun BRANCHBENCH in a fully controlled environment. For this demonstration, we instantiate the workloads over the CH-benCHmark schema [1], using five warehouses for the full macrobenchmark and the parameters in Table 1; Appendix A gives representative SQL templates for each workflow. Because neither system completed all full workflows within a two-hour timeout, we also use a mini configuration with one warehouse and fewer workers and steps, preserving each workflow’s per-step operation mix.

Full macrobenchmark statistics. Table 2 shows that neither demonstration system supports the full synthesized suite. Neon completes failure reproduction, but partially executes software development, data cleaning, and MCTS because hosted deployments limit concurrent live branches. In our experiments, Neon reached a 20 active branch limit despite running on its highest subscription tier; although this is documented as a soft limit, it was the effective limit encountered during benchmarking. Dolt completes software development, failure reproduction, and data cleaning, but MCTS and simulation become dominated by expensive branch-local reads through its content-addressed storage representation.

Figure 2 illustrates why the benchmark must measure both branch metadata and branch-local data operations. MCTS creates a deep speculative tree: Neon initially progresses quickly, then stalls when live-branch capacity is exhausted, while Dolt continues

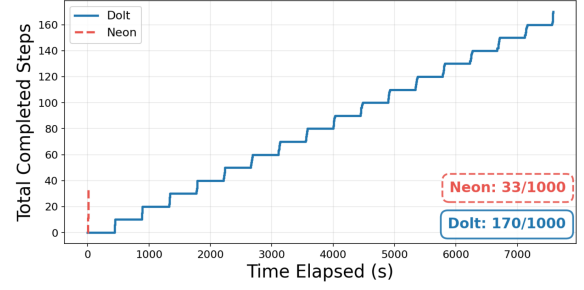


Figure 2: MCTS progress before timeout. Neon stalls at the live-branch limit; Dolt continues but slows on branch-local reads.

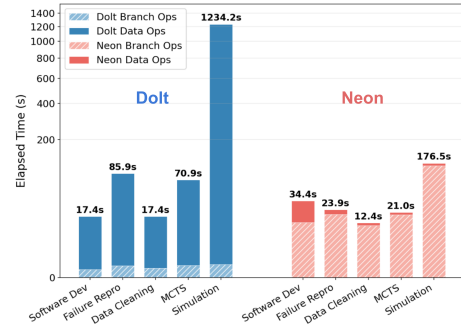


Figure 3: End-to-end latency by workflow on the mini configuration.

branching but fails to complete because joins and larger reads are expensive.

Storage behavior. We measure storage at workflow end or timeout. Table 3 shows workload-dependent overheads: Neon uses over 43× more storage on Software Dev because schema updates with backfills reduce page sharing, while Dolt uses more storage on Failure Repro and Data Cleaning because it retains pruned-branch history and materializes DEFAULT FALSE columns. Note that Neon’s storage values should be interpreted only as order-of-magnitude evidence. First, Neon and Dolt did not always complete the same number of workflow steps, so the final states are not strictly apples-to-apples. Second, Neon begins reporting storage metrics only after the run has been active for several minutes and then updates them hourly; for workflows that prune branches, this can miss transient storage growth or report storage after reclamation. Dolt was measured directly from the local database directory and is therefore more precise.

Mini macrobenchmark statistics. The mini macrobenchmarks let Neon and Dolt complete the same workloads. Figure 3 reports end-to-end latency by workflow, and Figure 4 decomposes latency by operation class. Across workflows, Neon spends most end-to-end time in branch management, while Dolt spends most time in branch-local data operations. Despite the network delay in our setup, Neon executes branch-local SQL much faster than Dolt on these workloads, but branch management is much slower than

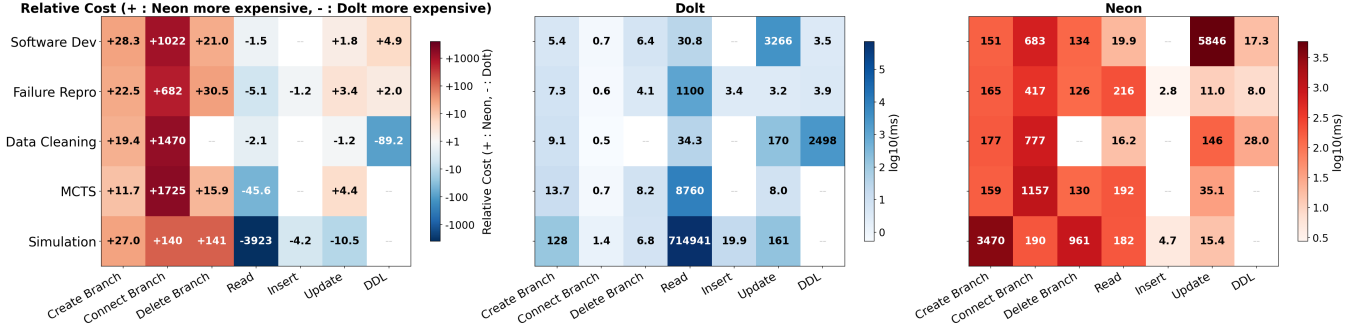


Figure 4: Median operation latency for Neon and Dolt on mini macrobenchmark configurations.

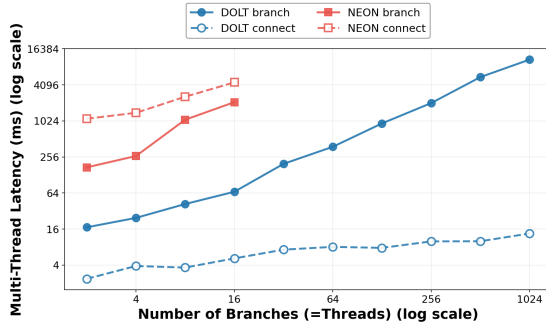


Figure 5: Branch creation and connection latency when creating x branches with x threads.

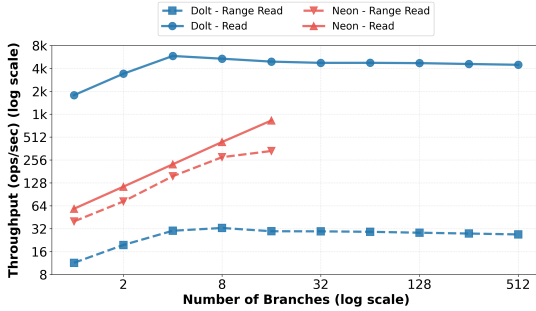


Figure 6: Read capacity as the number of branches increases (range size = 100).

Dolt's. Dolt's branch operations are fast metadata updates, but range reads and joins become 5–4000 $\times$ slower than Neon's. DDL also diverges: Dolt materializes default values during data-cleaning schema changes, whereas PostgreSQL-compatible engines treat them as metadata operations.

Microbenchmark breakdown. Microbenchmarks isolate the two components exposed by the macrobenchmarks. For branch metadata operations, Figure 5 measures the latency of creating and connecting to x branches using x client threads. Neon's latency is substantially higher because each branch must be registered

through the hosted control plane and attached to branch-specific compute, whereas Dolt implements branch creation and checkout by updating local metadata over versioned root hashes.

For branch-local data operations, Figure 6 measures range-read throughput as concurrent branches increase. Dolt's proly-tree traversal and query execution yield substantially lower range-read throughput than Neon, explaining why read-heavy macrobenchmarks become dominated by branch-local data operations. The same figure shows the complementary benefit of Neon's architecture: although allocating one compute per branch makes branch creation and connection more expensive, additional branches also add aggregate read capacity and improve branch isolation until the live-branch limit. Dolt branches cheaply, but all branches share one process and resource pool, so aggregate range-read throughput plateaus under concurrent branch access.

These drill-downs are representative examples chosen because they explain the most salient Neon–Dolt behaviors on our synthesized workflows. The microbenchmark suite also supports range updates, deletes, DDL operations, and new operation templates can be added with minimal effort.

4 CONCLUSION

BRANCHBENCH introduces an extensible benchmark framework for generating agentic database workloads from explicit branch topology, mutation, evaluation, and pruning parameters. By integrating parameterized macrobenchmarks with operation-level microbenchmarks, BRANCHBENCH enables systematic evaluation of how branchable database architectures match workload-specific branch lifecycles and branch-local data access patterns. Our evaluation of Neon and Dolt demonstrates that contemporary state-of-the-art branchable DBMSes remain insufficient for the proposed agentic workloads, exposing fundamental trade-offs between branch management overhead, aggregate read capacity, branch isolation, and branch-local query performance. Together, the macrobenchmark and microbenchmark results make BRANCHBENCH a practical tool for comparing existing branchable systems and an empirical foundation for designing database architectures that natively support agentic speculation.

REFERENCES

- [1] Richard Cole, Florian Funke, Leo Giakoumakis, Wey Guy, Alfons Kemper, Stefan Krompass, Harumi Kuno, Raghunath Nambiar, Thomas Neumann, Anisoara

- Patel, Meikel Poess, Kai-Uwe Sattler, Bernhard Seeger, Jan Takahashi, and Marek Wolski. 2011. Mixed Workload CH-benCHmark. In *Proceedings of the Fourth International Workshop on Testing Database Systems (DBTest)*. <https://doi.org/10.1145/1988842.1988850>
- [2] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *ACM Symposium on Cloud Computing*. <https://api.semanticscholar.org/CorpusID:2589691>
- [3] Djellel Eddine Difallah, Andrew Pavlo, Carlo Curino, and Philippe Cudré-Mauroux. 2013. OLTP-Bench: An Extensible Testbed for Benchmarking Relational Databases. *Proc. VLDB Endow.* 7 (2013), 277–288. <https://api.semanticscholar.org/CorpusID:2612270>
- [4] Mike Freedman. 2025. Fluid Storage: Forkable, Ephemeral, and Durable Infrastructure for the Age of Agents. <https://www.tigerdata.com/blog/fluid-storage-forkable-ephemeral-durable-infrastructure-age-of-agents>.
- [5] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercoovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, HaoWei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. 2026. Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces. *arXiv:2601.11868 [cs.SE]* <https://arxiv.org/abs/2601.11868>
- [6] Michael Stonebraker, Jim Frew, Kenn Gardels, and Jeff Meredith. 1993. The SEQUOIA 2000 storage benchmark. , 10 pages. <https://doi.org/10.1145/170036.170038>
- [7] Dolt Team. 2025. Dolt - Using Branches. <https://docs.dolthub.com/sql-reference/version-control/branches>.
- [8] Neon Team. 2026. Neon Branching - Branch your data the same way you branch your code. <https://neon.com/docs/introduction/branching>.
- [9] Xata Team. 2025. Xata Instant Branching: Copy-on-Write branching system for PostgreSQL databases. <https://xata.io/documentation/core-concepts/branching>.
- [10] Jiakai Xu, Tianle Zhou, Eugene Wu, and Kostis Kaffes. 2025. Toward Systems Foundations for Agentic Exploration. *SOSP SSA Workshop (2025)*.

A MACROBENCHMARK QUERIES

This section provides examples of the queries used in the macrobenchmark.

A.1 Software Development

Schema Change:

```
ALTER TABLE customer ADD COLUMN loyalty_tier_xx;
```

Data Mutation (backfill):

```
UPDATE customer SET loyalty_tier_xx = (CASE WHEN ...);
```

Read (evaluate quality):

```
SELECT loyalty_tier_xx, COUNT(*) FROM customer GROUP BY loyalty_tier_xx;
```

Compare (cross-branch):

```
SELECT loyalty_tier_xx, COUNT(*) AS tier_count,
AVG(c_ytd_payment) AS avg_payment
FROM customer GROUP BY loyalty_tier_xx;
```

A.2 Failure Reproduction

Schema Change (replay):

```
ALTER TABLE order_line ADD COLUMN X;
ALTER TABLE orders ADD/DROP COLUMN Y;
ALTER TABLE customer ADD COLUMN Z;
```

Data Mutation (replay):

```
INSERT INTO orders (...);
UPDATE customer SET (...);
DELETE FROM order_line WHERE (...);
```

Read (invariant check):

```
SELECT DISTINCT o.o_id FROM orders
JOIN order_line ON (...)
GROUP BY (...) HAVING (...);
```

A.3 Data Cleaning

Schema Change (aux column):

```
ALTER TABLE customer ADD COLUMN c_clean_xx DEFAULT FALSE;
```

Data Mutation (strategies):

```
UPDATE customer SET c_balance = 0 WHERE c_balance IS NULL;
DELETE FROM customer WHERE c_balance IS NULL;
```

Read (correctness):

```
SELECT COUNT(CASE WHEN c_balance < 0 THEN 1 END)
AS invalid FROM customer;
```

Compare (cross-branch):

```
SELECT COUNT(...) AS invalid,
MAX(c_ytd_payment)-MIN(c_ytd_payment) AS spread
FROM customer;
```

A.4 Monte Carlo Tree Search

Data Mutation:

```
UPDATE stock SET s_quantity = (...) WHERE (...);
```

Read (expected reward):

```
SELECT SUM(ol_amount) AS total_cost
FROM order_line JOIN warehouse ON (...);
```

A.5 Simulation

Data Mutation:

```
INSERT INTO orders (...);
UPDATE stock SET (...) WHERE (...);
INSERT INTO order_line (...);
```

Read (evaluate outcomes):

```
SELECT SUM(...) AS stockouts,
SUM(ol.ol_amount) AS total_cost
FROM stock JOIN order_line ON (...);
```

Compare (aggregate across branches):

```
SELECT AVG(stockouts), AVG(total_cost)
FROM all_branches.outcome_metrics;
```